Optimization algorithms timeline

- Gradient descent : 1847 Cauchy
- SGD : 1951 Robbins and Monro
- Momentum: 1964 Polyak
  1983 Nesterov → COLT
- Ada Grad: 2011 Duchi, Hazan, and Singer
- RMS Prop : 2012 Hinton → Online class
- Adam : 2014 Kingma and Ba → arXiv
- AdamW: 2017 Loschilov and Hutter → arXiv

Gradient Descent : No history

- $\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$
- Steep direction: Oscillation
  Shallow direction: No movement
- Zig zag path to the minima

SGD with momentum: History (Polyak)

- $V_t = \beta \cdot V_{t-1} + (1-\beta) \nabla_\theta L(\theta_t)$
- $\theta_{t+1} = \theta_t - \eta V_t$

- Exponentially weighted moving average of past gradients
- Smoother convergence, empirically
- No guarantee of being better than plain SGD
- Might miss sharp minima

Avoid overshooting with look-ahead (Nesterov)

- $V_t = \beta \cdot V_{t-1} + (1-\beta) \nabla_\theta L(\theta_t - \eta \beta V_{t-1})$
- $\theta_{t+1} \simeq \theta_t - \eta V_t$
- Polyak keeps pushing along the direction of inertia
- You might miss a sharp valley
- Polyak: Gradient → Momentum → Update
- Nesterov: Look ahead gradient → Momentum → Update

AdaGrad: Adaptive learning rate per parameter

- $g_{t,i} = \frac{\partial L(\theta_t)}{\partial \theta_i}$
- $S_{t,i} = S_{t-1,i} + g_{t,i}^2$
- $\theta_{t+1,i} = \theta_{t,i} - \eta_{t,i} \cdot g_{t,i}$
- $\eta_{t,i} = \frac{\eta}{\sqrt{S_{t,i}} + \epsilon}$

- Accummulate gradient over time
- Higher gradient, Lower learning rate
- Word2Vec used AdaGrad

RMS Prop: Maintain EWMA of second moment

- $S_{t,i} = \beta \left(S_{(t-1),i} \right) + (1-\beta)\, g^2_{t,i}$

- $\Theta_{t+1,i} = \Theta_{t,i} - \eta_{t,i} \cdot g_{t,i}$

- $\eta_{t,i} = \dfrac{\eta}{\sqrt{S_{t,i}} + \epsilon}$

- $S_{t,i}$ does not increase monotonically

Adam: Adaptive step size + Directional smoothing

- $g_{t,i} = \frac{\partial L(\theta_t)}{\partial \theta_i}$
- $m_{t,i} = \beta_1 m_{t-1,i} + (1-\beta_1) g_{t,i}$ ← Directional smoothing
- $v_{t,i} = \beta_2 v_{t-1,i} + (1-\beta_2) g_{t,i}^2$
- $\theta_{t+1,i} = \theta_{t,i} - \eta_{t,i} \cdot m_{t,i}$
- $\eta_{t,i} = \frac{\eta}{\sqrt{v_{t,i}} + \epsilon}$ ← Adaptive step size
- Typical values: $\beta_1 = 0.9$ $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Earlier updates are artificially small

- Boost initial updates using $(1-\beta_1^t)$, $(1-\beta_2^t)$

$$\hat{m}_{t,i} = \frac{m_{t,i}}{1-\beta_1^t}$$

$$\hat{v}_{t,i} = \frac{v_{t,i}}{1-\beta_2^t}$$

- For large t, $1-\beta_1^t \approx 1$, $1-\beta_2^t \approx 1$

Regularization keeps weights small

- $L_R =$ $L(\theta)$ + Regularization loss
- L2 regularization: $L_{total} = L_{data} + \frac{\lambda}{2}\|\theta\|_2^2$

  ↓ Squared L2 norm
- $\nabla L_{total} = \nabla L_{data} + \lambda\theta$
- SGD: $\theta_{t+1} = \theta_t - \eta\left(\nabla L_{data_t} + \lambda\theta_t\right)$

  $= (1-\eta\lambda)\theta_t - \eta \nabla L_{data_t}$
- Every parameter experiences uniform shrinking or decay

Regularization avoids overfitting

- Overfitting: Works on training data
  Fails on test data
- Large weights → Model sensitive to small changes in input
- Small weights → Smoother decision boundaries

Vanilla Adam fails to provide uniform shrinking

- $g_t = g_t + \lambda \theta_t$
- $\theta_{t+1,i} = \theta_{t+1,i} - \eta_{t,i} \cdot m_{t,i}$
- $\eta_{t,i}$ is different for each parameter

AdamW : Decoupled weight decay

- $$\theta_{t+1,i} = \theta_{t,i} - \eta_{t,i} \cdot \hat{m}_{t,i} - \eta \cdot \lambda \cdot \theta_{t,i}$$
- Better generalization

Memory footprint of optimizer

- Number of parameters: X
- Gradient: X
- First moment (m_t) : X
- Second moment (v_t) : X
- Total: 4X
- Imagine a 100B model
- How can we reduce memory footprint?
  - Parameters and Gradient, cannot compromize
  - First moment can be ignored (no momentum)
  - Second moment can be approximated

AdaFactor makes Adam memory efficient

- Gradients are correlated rowise and columnwise
- Value of a gradient is typically a function of its row number and column number
- We actually don't care about exact values of $V_{t,i}$
- We only need to maintain relative step size across the parameters
- $$V[m][n] = \frac{r_m \cdot c_n}{\text{mean}(g^2)}$$